

Assignment 6: Distributed Systems (Middleware)

Start Assignment

- Due Dec 6 by 11:59pm
- Points 100
- Submitting a text entry box, a website url, or a file upload

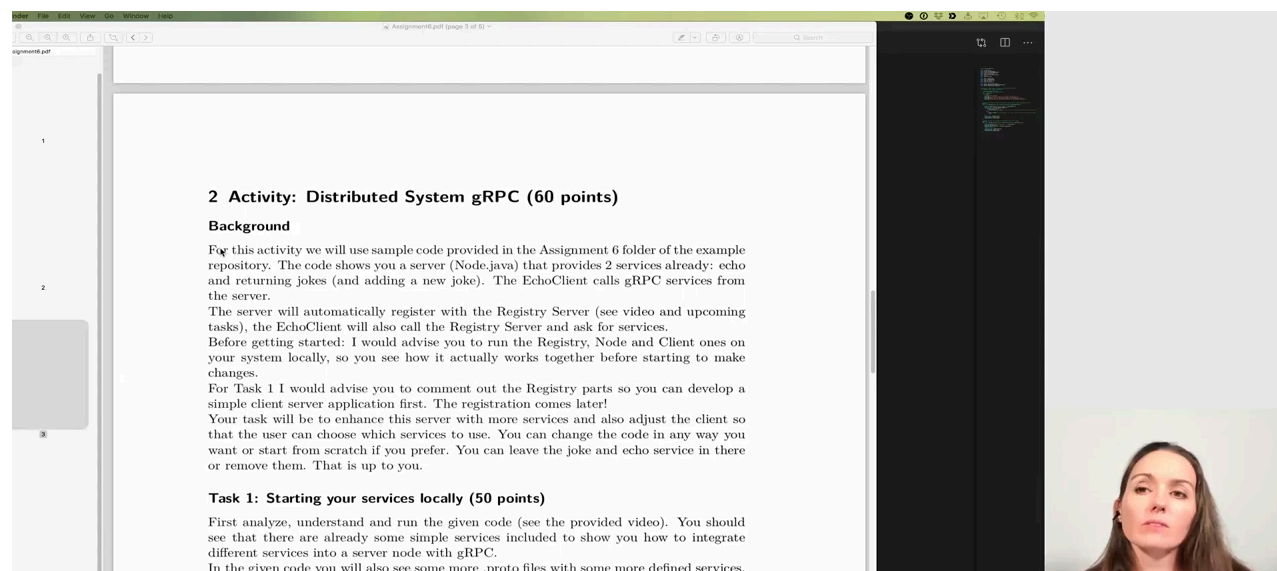
Assignment (<https://canvas.asu.edu/courses/197725/files/97975365?wrap=1>)

Starter Code (<https://canvas.asu.edu/courses/197725/files/90125640?wrap=1>) 
(https://canvas.asu.edu/courses/197725/files/90125640/download?download_frd=1)

(<https://canvas.asu.edu/courses/197725/files/90125241/download?wrap=1>) NO SUBMISSIONS LATER THAN Dec 8th even with late pass!!!

Some videos, these are optional but I hope they help you see how the code works and how it runs. rom slightly different versions, I am planning on updating them BUT it is almost the same

Activity 1 (this used to be activity 2 when I recorded this so please do not be too confused, the general project layout is the same)



The screenshot shows a video call interface. On the left, a PDF document titled 'Assignment 6' is open, displaying the following text:

2 Activity: Distributed System gRPC (60 points)

Background

For this activity we will use sample code provided in the Assignment 6 folder of the example repository. The code shows you a server (Node.java) that provides 2 services already: echo and returning jokes (and adding a new joke). The EchoClient calls gRPC services from the server.

The server will automatically register with the Registry Server (see video and upcoming tasks), the EchoClient will also call the Registry Server and ask for services.

Before getting started: I would advise you to run the Registry, Node and Client ones on your system locally, so you see how it actually works together before starting to make changes.

For Task 1 I would advise you to comment out the Registry parts so you can develop a simple client server application first. The registration comes later!

Your task will be to enhance this server with more services and also adjust the client so that the user can choose which services to use. You can change the code in any way you want or start from scratch if you prefer. You can leave the joke and echo service in there or remove them. That is up to you.

Task 1: Starting your services locally (50 points)

First analyze, understand and run the given code (see the provided video). You should see that there are already some simple services included to show you how to integrate different services into a server node with gRPC.

In the given code you will also see some more .proto files with some more defined services.

On the right side of the video call, a person's face is visible in the bottom right corner.

13:17

Code Walkthrough (this is from an older version of the code, it is almost the same though)

Assign 6 Code walk through



Running the Program

Running the program



AWS grPC

Criteria	Ratings		Pts
Task 1: all the gradle tasks work	3 pts Full Marks	0 pts No Marks	3 pts
Task 1: 2 services from the given ones are started to be implemented 2 for each The classes are present (e.g. FlowerImpl, FlowImpl) might be called differently	4 pts Full Marks	0 pts No Marks	4 pts
Task 1 Service 1: All methods are started to be implemented Methods are at least all there and started. 0.5 per method	2 pts Full Marks	0 pts No Marks	2 pts
Task 1 Service 1: All methods from the proto are implemented in the Impl class Methods seem to be implemented correctly	3 pts Full Marks	0 pts No Marks	3 pts
Task 1 Service 1: The Client collects the information from the user well to call the service Here run without -auto and see that the client asks well for all the info and that the requests work.	3 pts Full Marks	0 pts No Marks	3 pts
Task 1 Service 1: The services work correctly and handle things as specified in the proto Our Unit Tests pass (5 points) Their Unit Tests or client functionality works (5 points)	10 pts Full Marks	0 pts No Marks	10 pts
Task 1 Service 2: All methods are started to be implemented	2 pts Full Marks	0 pts No Marks	2 pts
Task 1 Service 2: All methods from the proto are implemented in the GRPC class	3 pts Full Marks	0 pts No Marks	3 pts
Task 1 Service 2: The Client collects the information from the user well to call the service	3 pts Full Marks	0 pts No Marks	3 pts

Criteria	Ratings		Pts
Task 1 Service 2: The services work correctly and handle things as specified in the proto Our Unit Tests pass (5 points) Their Unit Tests or client functionality works (5 points)	10 pts Full Marks	0 pts No Marks	10 pts
Task 1: Client asks user which service the user wants to use	3 pts Full Marks	0 pts No Marks	3 pts
Task 1: Client asks for the appropriate input	2 pts Full Marks	0 pts No Marks	2 pts
Task 1: Client does not crash with wrong input	3 pts Full Marks	0 pts No Marks	3 pts
Task 1: Auto run or Unit Tests are implemented and work correctly and show that things work as intended	4 pts Full Marks	0 pts No Marks	4 pts
Task 1: When wrong input is given or wrong client requests are send the server does not crash and handles the errors well.	5 pts Full Marks	0 pts No Marks	5 pts
Task 2: proto file is well written and compiles gradle generateProto works	5 pts Full Marks	0 pts No Marks	5 pts
Task 2: Proto fulfills at least 2 of the criteria Service allows at least 2 different requests • Each request needs at least 1 input • Response returns different data for different requests • Response returns a repeated field • Data is held persistent on the server	5 pts Full Marks	0 pts No Marks	5 pts
Task 2: Class for the service is created and implements the requests	5 pts	0 pts	5 pts

Criteria	Ratings		Pts
	Full Marks	No Marks	
Task 2: Client lets user choose this new service	3 pts Full Marks	0 pts No Marks	3 pts
Task 2: Client and server work well together to let the user use the service. This includes no crashes on client for wrong input -2 This includes no crashes on client for wrong input -2 Asking for the correct input -3 Showing the results well -3 Overall good user interface -2	12 pts Full Marks	0 pts No Marks	12 pts
Task 3: New Client class is created	1 pts Full Marks	0 pts No Marks	1 pts
Task 3: New Client can be called through Gradle	2 pts Full Marks	0 pts No Marks	2 pts
Task 3: Client contacts registry and prints available services	3 pts Full Marks	0 pts No Marks	3 pts
Task 3: Client chooses a services and is asked for input	2 pts Full Marks	0 pts No Marks	2 pts
Task 3: Client sends request to one of the available nodes that implements the services	2 pts Full Marks	0 pts No Marks	2 pts
Task 3.2: Put your node online	2.5 pts Full Marks	0 pts No Marks	2.5 pts

Criteria	Ratings		Pts
Task 3.2L Test at least one other node	2.5 pts Full Marks	0 pts No Marks	2.5 pts
Total Points: 105			